

- Cada vez que se define un conjunto de valores, se está definiendo un nuevo tipo de dato
- En un tipo de dato se define los valores, representaciones y operaciones sobre el mismo
- La representación de los datos es bastante compleja
- Para cambiar la representación de los datos se lleva a cabo una tarea llamada **Abstracción de datos**



- La abstracción de datos divide en dos los tipos de datos:
interface e **implementación**
- La interfaz de un tipo de datos dice que representa el tipo de dato, sus operaciones y las propiedades de dichas operaciones
- La implementación proporciona una representación específica y un código para las operaciones que hacen uso de esa representación.



- Un tipo de dato que es abstracto se denomina Tipo Abstracto de Dato (TAD).
- El resto del programa fuera del tipo de dato, llamado el cliente del tipo de dato, manipula el nuevo dato solo mediante las operaciones especificadas en la interfaz
- El código del cliente es independiente de la representación si manipula los valores del tipo de dato solo a través de los procedimientos en su interfaz.



- Todo lo relacionado sobre el dato representado debe estar en el código de la implementación
- La parte más importante de la implementación es la especificación de como los datos son representados
- Se utiliza la implementación $\lceil v \rceil$ para la representación de dato v



Representación de números naturales:

$$(\text{zero}) = [0]$$

$$(\text{is-zero? } [n]) = \begin{cases} \text{\#t} & n = 0 \\ \text{\#f} & n \neq 0 \end{cases}$$

$$(\text{successor } [n]) = [n + 1] \quad (n \geq 0)$$

$$(\text{predecessor } [n + 1]) = [n] \quad (n \geq 0)$$



Representación de números naturales:

- En la anterior especificación no se indica cómo se representan los números naturales
- A partir de la especificación se pueden escribir programas para la manipulación de los datos, sin importar su representación



Representación de números naturales:

```
(define plus
  (lambda (x y)
    (if (is-zero? x)
        y
        (successor (plus (predecessor x) y))
    )
  )
)
```

Satisface $(\text{plus } [x] [y]) = [x + y]$



Tres posibles representaciones para los enteros no negativos:

- 1 *Representación Unaria*: Donde un entero no negativo n es representado por una lista de n símbolos '#t'.

$$[0] = ()$$
$$[n + 1] = (\text{cons } \text{\#t } [n])$$

En esta representación, se satisface la especificación escribiendo:

```
(define zero '())  
(define iszero? null?)  
(define succ (lambda (n) (cons #t n)))  
(define pred cdr)
```



- 2 *Representación de Números de Scheme*: Se usa la representación interna de números de Scheme.

$$\lceil n \rceil = n$$

Se definen las entidades como:

```
(define zero 0)
(define iszero? zero?)
(define succ (lambda (n) (+ n 1)))
(define pred (lambda (n) (- n 1)))
```



- 3 *Representación Bignum*: Los números son representados en base N , para algún entero grande N . Dicha representación es una lista que consiste de números entre 0 y $N - 1$.

$$[n] = \begin{cases} () & n = 0 \\ (\text{cons } r [q]) & n = qN + r, 0 \leq r < N \end{cases}$$

Luego si $N = 16$, entonces:

$$\begin{aligned} [33] &= (1 \ 2) && ((1 \times 16^0) + (2 \times 16^1)) \\ [258] &= (2 \ 0 \ 1) && ((2 \times 16^0) + (0 \times 16^1) + (1 \times 16^2)). \end{aligned}$$



- Cuando es usada la abstracción de datos, los programas tienen la propiedad de independencia de representación.
- Se presentan algunas estrategias para representar tipos de datos. Se ilustran estas estrategias usando el tipo de dato *ambiente*.
- Un ambiente asocia un valor con cada elemento de un conjunto finito de variables.
- Un ambiente puede ser usado para asociar las variables con sus valores en la implementación de un lenguaje de programación.



- Las variables pueden ser representadas de cualquier manera, siempre y cuando sea posible chequear la igualdad entre dos variables.
- Las variables se pueden representar mediante símbolos, cadenas , referencias en una tabla hash o incluso mediante números.



- Un ambiente es una función cuyo dominio es un conjunto finito de variables y cuyo rango es el conjunto de todos los valores de Scheme.
- De acuerdo a la notación matemática, los ambientes representan todos los conjuntos de la forma $\{(s_1, v_1), \dots, (s_n, v_n)\}$, donde los s_i son símbolos diferentes y los v_i son valores de Scheme.



La interfaz del tipo de dato ambiente tiene tres procedimientos:

<code>(empty-env)</code>	$= [\emptyset]$
<code>(apply-env $[f]$ s)</code>	$= f(s)$
<code>(extend-env var v $[f]$)</code>	$= [g],$

donde $g(s') = \begin{cases} v & \text{si } s' = var \\ f(s') & \text{de otra forma} \end{cases}$



- El procedimiento `empty-env` debe producir una representación del ambiente vacío.
- El procedimiento `apply-env` aplica una representación de un ambiente a un argumento.
- El procedimiento `(extend-env var val env)` produce un nuevo ambiente que se comporta como `env`, excepto que su valor en el símbolo `var` es `val`.

```
(define e
  (extend-env 'd 6
    (extend-env 'y 8
      (extend-env 'x 7
        (extend-env 'y 14
          (empty-env)))))))
```



Representación como estructura de datos:

- Cada ambiente puede ser construido mediante una expresión en la siguiente gramática:

$$\begin{aligned}\langle \text{env-exp} \rangle & ::= (\text{empty-env}) \\ & ::= (\text{extend-env } \langle \text{identificador} \rangle \\ & \quad \langle \text{scheme-value} \rangle \langle \text{env-exp} \rangle)\end{aligned}$$

- De acuerdo a esta gramática los ambientes pueden ser representados como listas en Scheme.



Representación como estructura de datos:

empty-env: $() \rightarrow Env$

```
(define empty-env  
  (lambda () (list 'empty-env)))
```

extend-env: $Var \times SchemeValue \times Env \rightarrow Env$

```
(define extend-env  
  (lambda (var val env)  
    (list 'extend-env var val env)))
```



Representación como estructura de datos:

apply-env: $Env \times Var \rightarrow SchemeValue$

```
(define apply-env
  (lambda (env search-var)
    (cond ((eqv? (car env) 'empty-env)
           (report-no-binding-found search-var))
          ((eqv? (car env) 'extend-env)
           (let ((saved-var (cadr env))
                 (saved-val (caddr env))
                 (saved-env (cadddr env)))
             (if (eqv? search-var saved-var)
                 saved-val
                 (apply-env saved-env search-var))))
          (else (report-invalid-env env))))))
```



Representación Procedimental:

- La interfaz del tipo de dato ambiente tiene una propiedad importante: ella tiene exactamente una entidad *observadora* `apply-env`.
- Esto permite representar un ambiente como un procedimiento que toma una variable y retorna su valor asociado.
- Se define `empty-env` y `extend-env` de tal manera que retornan un procedimiento que al ser aplicado devuelve el valor asociado a la variable en el ambiente.



Representación Procedimental:

empty-env: $() \rightarrow Env$

```
(define empty-env
  (lambda ()
    (lambda (search-var)
      (report-no-binding-found search-var))))
```

extend-env: $Var \times SchemeValue \times Env \rightarrow Env$

```
(define extend-env
  (lambda (saved-var saved-val saved-env)
    (lambda (search-var)
      (if (eqv? search-var saved-var)
          saved-val
          (apply-env saved-env search-var)))))
```



Representación Procedimental:

apply-env: $Env \times Var \rightarrow SchemeValue$

```
(define apply-env  
  (lambda (env search-var)  
    (env search-var)))
```



Hasta el momento hemos visto como definir tipos de datos recursivos mediante diferentes métodos. Por ejemplo:

```
<Lc-exp> ::= <var-exp>      (<identifier>)  
          ::= <lambda-exp>   (lambda (<identifier>) <Lc-exp>)  
          ::= <app-exp>      (<Lc-exp> <Lc-exp>)
```



Por ejemplo para una representación basada en listas:

```
;;x  
'x  
;;(lambda (x) x)  
(list 'lambda x 'x)  
;;;( (lambda (x) (lambda (y) (x y))) x)  
(list 'lambda 'x (list 'lambda 'y (list 'x 'y)))
```



Así mismo, hemos intentado manipular y operar tipos de datos recursivos. Por ejemplo, para las expresiones del cálculo λ se han definido procedimientos como `occurs-free?`:

```
(define occurs-free?
  (lambda (var exp)
    (cond
      ;;Caso var-exp
      [(symbol? exp) (eqv? var exp)]
      ;;Caso lambda-exp
      [(eqv? (car exp) 'lambda)
       (and (not (eqv? var (car (cadr exp))))
            (occurs-free? var (caddr exp)))]
      ;;Caso app-exp
      [else
       (or (occurs-free? var (car exp))
           (occurs-free? var (cadr exp)))])))
```



- No obstante, esta definición de `occurs-free?` es difícil de leer (es difícil decir que `(car (cadr exp))` se refiere a la declaración de una variable en una expresión lambda o que `(caddr exp)` se refiere a su cuerpo).
- Esta definición establece cierta dependencia con la implementación de las expresiones del cálculo λ como listas.



- Se puede mejorar estos aspectos introduciendo interfaces para la creación y manipulación de datos de un cierto tipo.
- Una interfaz para un tipo de dato consta de procedimientos *constructores* y procedimientos *observadores*.
- Los procedimientos observadores pueden ser predicados o extractores.



Para las expresiones del cálculo λ tenemos la siguiente interfaz:

■ Constructores:

var-exp : $Var \rightarrow Lc-exp$

lambda-exp : $Var \times Lc-exp \rightarrow Lc-exp$

app-exp : $Lc-exp \times Lc-exp \rightarrow Lc-exp$

■ Predicados:

var-exp? : $Lc-exp \rightarrow Bool$

lambda-exp? : $Lc-exp \rightarrow Bool$

app-exp? : $Lc-exp \rightarrow Bool$



■ Extractores:

<code>var-exp->var</code>	$: Lc-exp \rightarrow Var$
<code>lambda-exp->bound-var</code>	$: Lc-exp \rightarrow Var$
<code>lambda-exp->body</code>	$: Lc-exp \rightarrow Lc-exp$
<code>app-exp->rator</code>	$: Lc-exp \rightarrow Lc-exp$
<code>app-exp->rand</code>	$: Lc-exp \rightarrow Lc-exp$



Ahora es posible escribir una versión de occurs-free? que depende solo de la interfaz.

```
(define occurs-free?
  (lambda (search-var exp)
    (cond
      ((var-exp? exp) (eqv? search-var (var-exp->var exp)))
      ((lambda-exp? exp)
       (and
        (not (eqv? search-var (lambda-exp->bound-var exp)))
        (occurs-free? search-var (lambda-exp->body exp))))
      (else
       (or
        (occurs-free? search-var (app-exp->rator exp))
        (occurs-free? search-var (app-exp->rand exp)))))))
```



- La interfaz para el tipo de dato ambiente consta de los constructores `empty-env` y `extended-env` y del procedimiento observador `apply-env`.
- La interfaz para el tipo de dato lista consta de los constructores `empty-list` y `cons`, de los procedimientos extractores `car` y `cdr` y del predicado `list?`.



Receta general para el diseño de interfaces de datos recursivos

- 1 Incluir un constructor para cada clase de dato (regla de producción) en el tipo de dato.
- 2 Incluir un predicado para cada clase de dato en el tipo de dato.
- 3 Incluir un extractor para cada pieza de dato pasada a un constructor del tipo de dato.

